

Department of Computer Engineering
University of Peradeniya
CO523 - Programming Languages
Lab 11: Memory Management, Garbage Collection, and Runtime Systems

E/20/449

WIJEWARDHANA A.R.S.S.

Task 01

Implementation Code

```
import gc

class LabObject:
    def __init__(self, name):
        self.name = name
        print(f"Object '{self.name}' created.")

    def __del__(self):
        print(f"Object '{self.name}' is being deleted from memory.")

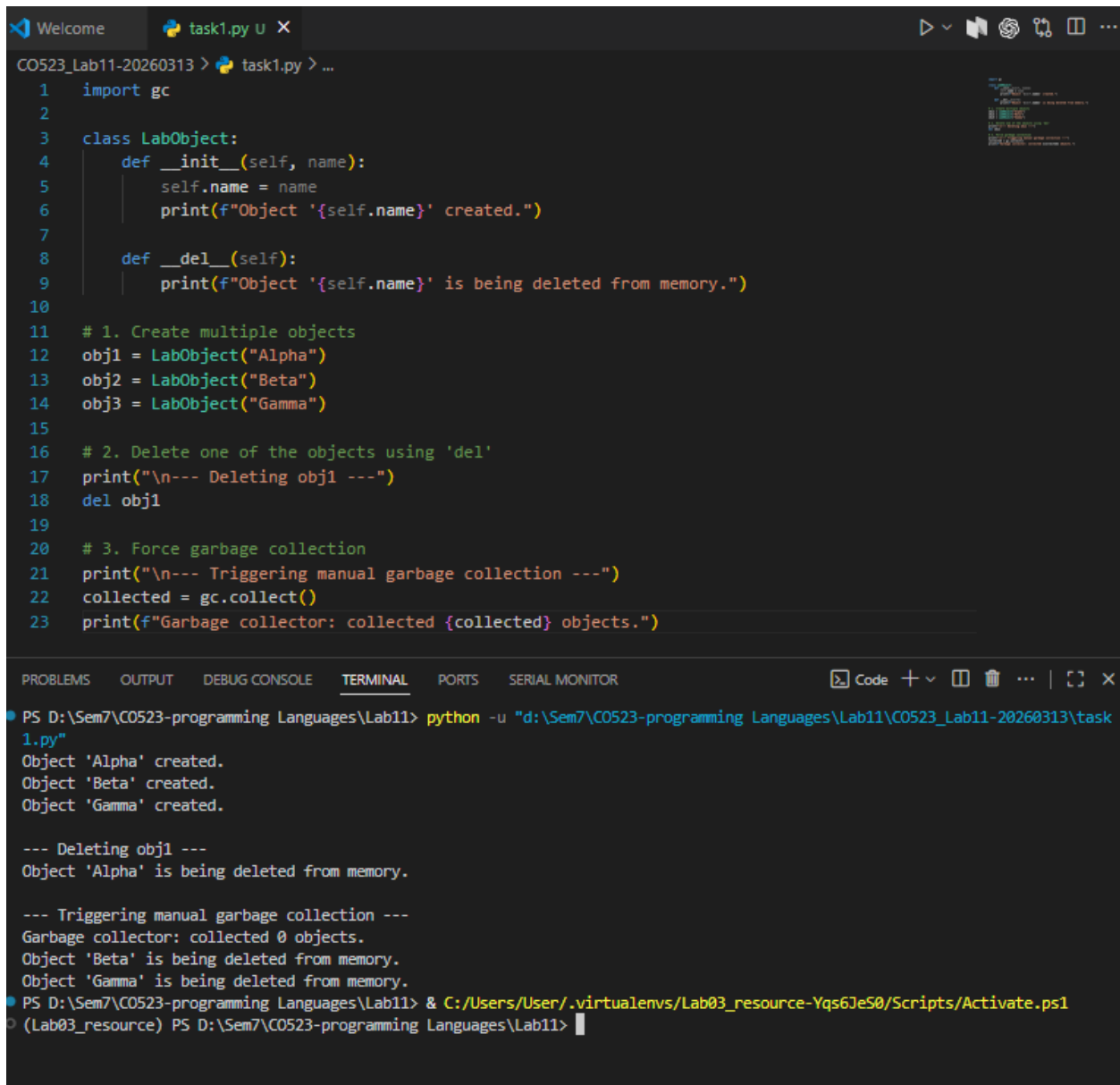
# 1. Create multiple objects
obj1 = LabObject("Alpha")
obj2 = LabObject("Beta")
obj3 = LabObject("Gamma")

# 2. Delete one of the objects using 'del'
print("\n--- Deleting obj1 ---")
del obj1

# 3. Force garbage collection
print("\n--- Triggering manual garbage collection ---")
collected = gc.collect()
print(f"Garbage collector: collected {collected} objects.")
```

Explanation of Steps

1. Object Creation: When `LabObject("Alpha")` is called, memory is allocated in the heap, and the `__init__` method prints a creation message.
2. Using `del`: The `del obj1` command removes the name "obj1" from the current scope. This decreases the reference count of the "Alpha" object.
3. Garbage Collection: While Python often cleans up objects with a reference count of zero immediately, calling `gc.collect()` forces the Generational Garbage Collector to scan for any remaining unreachable objects (like those in cyclic references) and clear them.



The screenshot shows a Python IDE with a code editor and a terminal. The code editor displays a Python script named `task1.py` that defines a `LabObject` class and demonstrates object creation, deletion, and garbage collection. The terminal shows the output of running the script, including the creation and deletion of objects and the results of a manual garbage collection.

```
CO523_Lab11-20260313 > task1.py > ...
1  import gc
2
3  class LabObject:
4      def __init__(self, name):
5          self.name = name
6          print(f"Object '{self.name}' created.")
7
8      def __del__(self):
9          print(f"Object '{self.name}' is being deleted from memory.")
10
11 # 1. Create multiple objects
12 obj1 = LabObject("Alpha")
13 obj2 = LabObject("Beta")
14 obj3 = LabObject("Gamma")
15
16 # 2. Delete one of the objects using 'del'
17 print("\n--- Deleting obj1 ---")
18 del obj1
19
20 # 3. Force garbage collection
21 print("\n--- Triggering manual garbage collection ---")
22 collected = gc.collect()
23 print(f"Garbage collector: collected {collected} objects.")
```

Terminal Output:

```
PS D:\Sem7\C0523-programming Languages\Lab11> python -u "d:\Sem7\C0523-programming Languages\Lab11\C0523_Lab11-20260313\task
1.py"
Object 'Alpha' created.
Object 'Beta' created.
Object 'Gamma' created.

--- Deleting obj1 ---
Object 'Alpha' is being deleted from memory.

--- Triggering manual garbage collection ---
Garbage collector: collected 0 objects.
Object 'Beta' is being deleted from memory.
Object 'Gamma' is being deleted from memory.
PS D:\Sem7\C0523-programming Languages\Lab11> & C:/Users/User/.virtualenvs/Lab03_resource-Yqs6JeS0/Scripts/Activate.ps1
(Lab03_resource) PS D:\Sem7\C0523-programming Languages\Lab11>
```

Explanation and Analysis

- We initialized three instances of LabObject. As shown in the terminal, each call to the class constructor triggered the `__init__` method, allocating space in the heap memory for "Alpha", "Beta", and "Gamma".
- When we executed `del obj1`, the reference to the "Alpha" object was removed. Since the reference count for "Alpha" dropped to zero immediately, Python's memory management automatically deallocated it, which we confirmed by the message "Object 'Alpha' is being deleted from memory" appearing right after the delete command.
- We invoked `gc.collect()` to force a scan of the memory generations. The output shows "collected 0 objects" because "Alpha" had already been handled by reference counting, and there were no cyclic references to clean up.
- The objects "Beta" and "Gamma" were deleted only when the program finished execution and their respective references (`obj2` and `obj3`) went out of scope.

Task 01 Question : Explain the output observed when the object is deleted.

When we delete the object using the `del` keyword, the reference count of that specific object reaches zero. In Python, this triggers immediate deallocation through the reference counting mechanism. We observed this because the object's destructor message appeared in the terminal immediately after the `del` command was issued, rather than waiting for the manual `gc.collect()` call or the end of the script. This demonstrates that while Python has a generational garbage collector for complex cycles, it uses reference counting for efficient, real-time memory reclamation of simple objects.

Task 02

Implementation Code

```
import gc
import sys

class Student:
    def __init__(self, name):
        self.name = name
        print(f"Student object '{self.name}' created.")

    def __del__(self):
        print(f"Student object '{self.name}' is being removed from memory.")

# 1. Create a list containing several objects
```

```
print("--- Initializing List ---")
student_list = [Student("Kamal"), Student("Nimal"), Student("Sunil")]

# 2. Show reference count (optional but helpful for observation)
# sys.getrefcount returns actual count + 1 (temporary reference)
print(f"Reference count for student_list[0]: {sys.getrefcount(student_list[0])}")

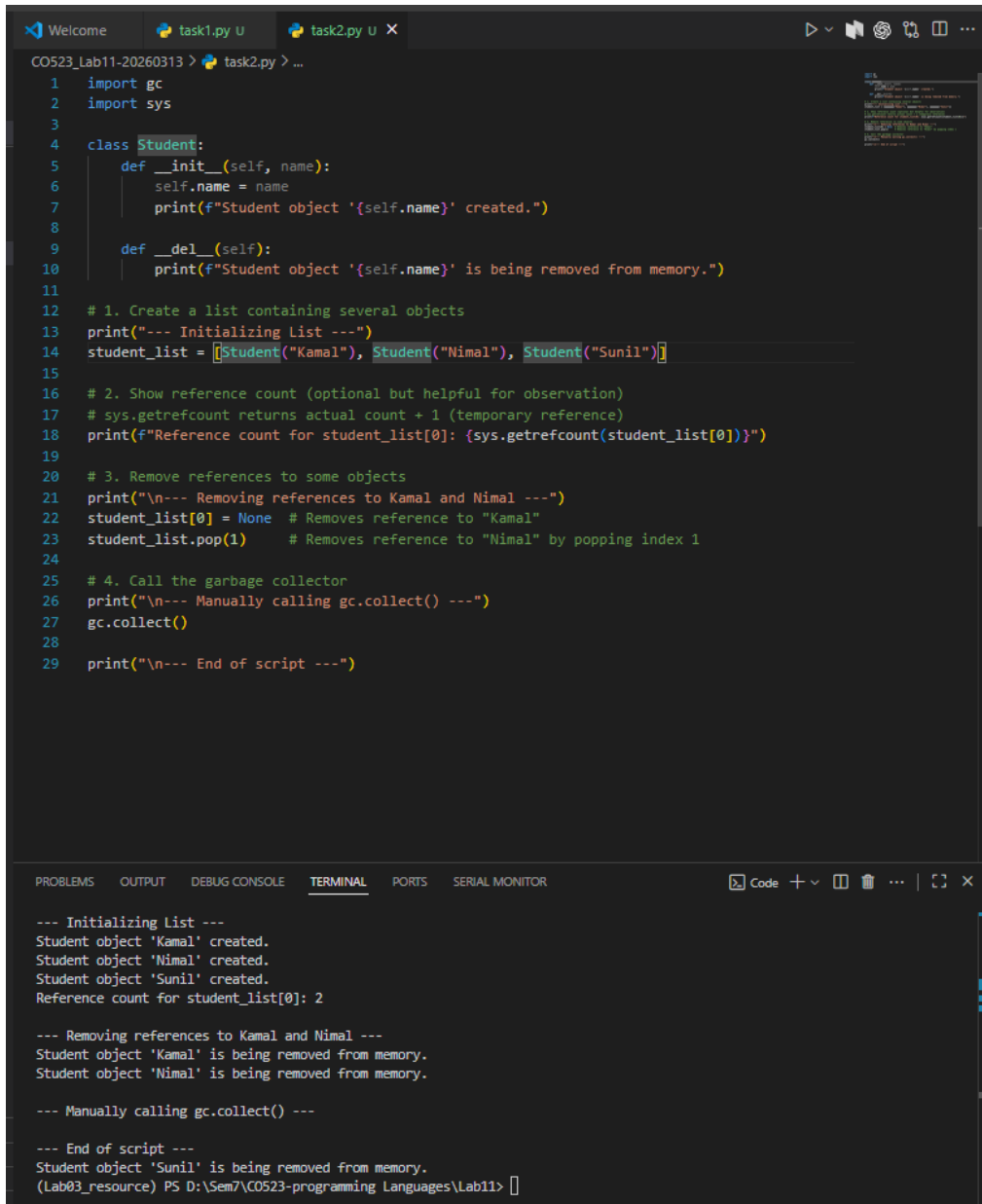
# 3. Remove references to some objects
print("\n--- Removing references to Kamal and Nimal ---")
student_list[0] = None # Removes reference to "Kamal"
student_list.pop(1)     # Removes reference to "Nimal" by popping index 1

# 4. Call the garbage collector
print("\n--- Manually calling gc.collect() ---")
gc.collect()

print("\n--- End of script ---")
```

Explanation of Steps

- **List Storage:** When objects are placed in a list, the list holds a reference to each object in heap memory.
- **Removing References:** By setting a list index to `None` or using `pop()`, we break the link between the variable and the object.
- **Eligibility:** An object becomes eligible for garbage collection the moment there are no active references pointing to it.
- **Manual Trigger:** While Python's `gc` runs automatically based on allocation thresholds, `gc.collect()` forces an immediate check of all generations.



```
1 import gc
2 import sys
3
4 class Student:
5     def __init__(self, name):
6         self.name = name
7         print(f"Student object '{self.name}' created.")
8
9     def __del__(self):
10        print(f"Student object '{self.name}' is being removed from memory.")
11
12 # 1. Create a list containing several objects
13 print("--- Initializing List ---")
14 student_list = [Student("Kamal"), Student("Nimal"), Student("Sunil")]
15
16 # 2. Show reference count (optional but helpful for observation)
17 # sys.getrefcount returns actual count + 1 (temporary reference)
18 print(f"Reference count for student_list[0]: {sys.getrefcount(student_list[0])}")
19
20 # 3. Remove references to some objects
21 print("\n--- Removing references to Kamal and Nimal ---")
22 student_list[0] = None # Removes reference to "Kamal"
23 student_list.pop(1)    # Removes reference to "Nimal" by popping index 1
24
25 # 4. Call the garbage collector
26 print("\n--- Manually calling gc.collect() ---")
27 gc.collect()
28
29 print("\n--- End of script ---")
```

```
--- Initializing List ---
Student object 'Kamal' created.
Student object 'Nimal' created.
Student object 'Sunil' created.
Reference count for student_list[0]: 2

--- Removing references to Kamal and Nimal ---
Student object 'Kamal' is being removed from memory.
Student object 'Nimal' is being removed from memory.

--- Manually calling gc.collect() ---

--- End of script ---
Student object 'Sunil' is being removed from memory.
(Lab03_resource) PS D:\Sem7\COS23-programming Languages\Lab11>
```

Output Explanation

The terminal output shows the exact lifecycle of objects in the Python heap

- Three Student objects ("Kamal", "Nimal", "Sunil") are created and stored in heap memory. The variable `student_list` holds references to these objects.
- The count for `student_list[0]` is 2. This is because one reference is held by the list itself, and the second is a temporary reference created by the `sys.getrefcount()` function during its execution.
- Removing References:
 - When we set `student_list[0] = None`, the reference to "Kamal" is removed.
 - When we use `student_list.pop(1)`, the reference to "Nimal" is removed.

- Notice that "Kamal" and "Nimal" are removed before we manually call `gc.collect()`. This is because Python's Reference Counting strategy deallocates objects immediately once their reference count drops to zero.
- "Sunil" is only removed at the very end because it remains reachable within the list until the program terminates.

Task 02 Question: Observe and explain when the objects are removed from memory.

The objects are removed from memory at two different stages depending on their reference status

1. Immediate Removal via Reference Counting: Objects ("Kamal" and "Nimal") are removed as soon as their references are reassigned or popped. This happens because Python's primary memory management strategy is reference counting, which triggers deallocation the moment the count reaches zero.
2. Removal at Program Termination: The remaining object ("Sunil") is removed when the script finishes execution. At this point, the `student_list` variable itself is destroyed, causing the reference count for "Sunil" to drop to zero, finally triggering the garbage collector.

Task 03

Implementation Code

We will create a Student class and a main class to manipulate object references. We use `System.gc()` to request a collection cycle.

```
// Student class with a finalize method to observe cleanup
class Student {
    String name;

    Student(String name) {
        this.name = name;
        System.out.println("Student object '" + this.name + "' created in heap.");
    }

    // This method is called by the GC before destroying the object
    @Override
    protected void finalize() throws Throwable {
        System.out.println("Garbage Collector is removing '" + this.name + "'.");
    }
}

public class GarbageExample {
    public static void main(String[] args) {
```

```

// 1. Create at least three objects
Student s1 = new Student("Kamal");
Student s2 = new Student("Nimal");
Student s3 = new Student("Sunil");

// 2. Remove references to one or two objects using null
System.out.println("\n--- Setting s1 and s2 to null ---");
s1 = null; // s1 becomes eligible for GC
s2 = null; // s2 becomes eligible for GC

// 3. Call System.gc()
System.out.println("--- Requesting Garbage Collection ---");
System.gc(); // Request JVM to run garbage collector

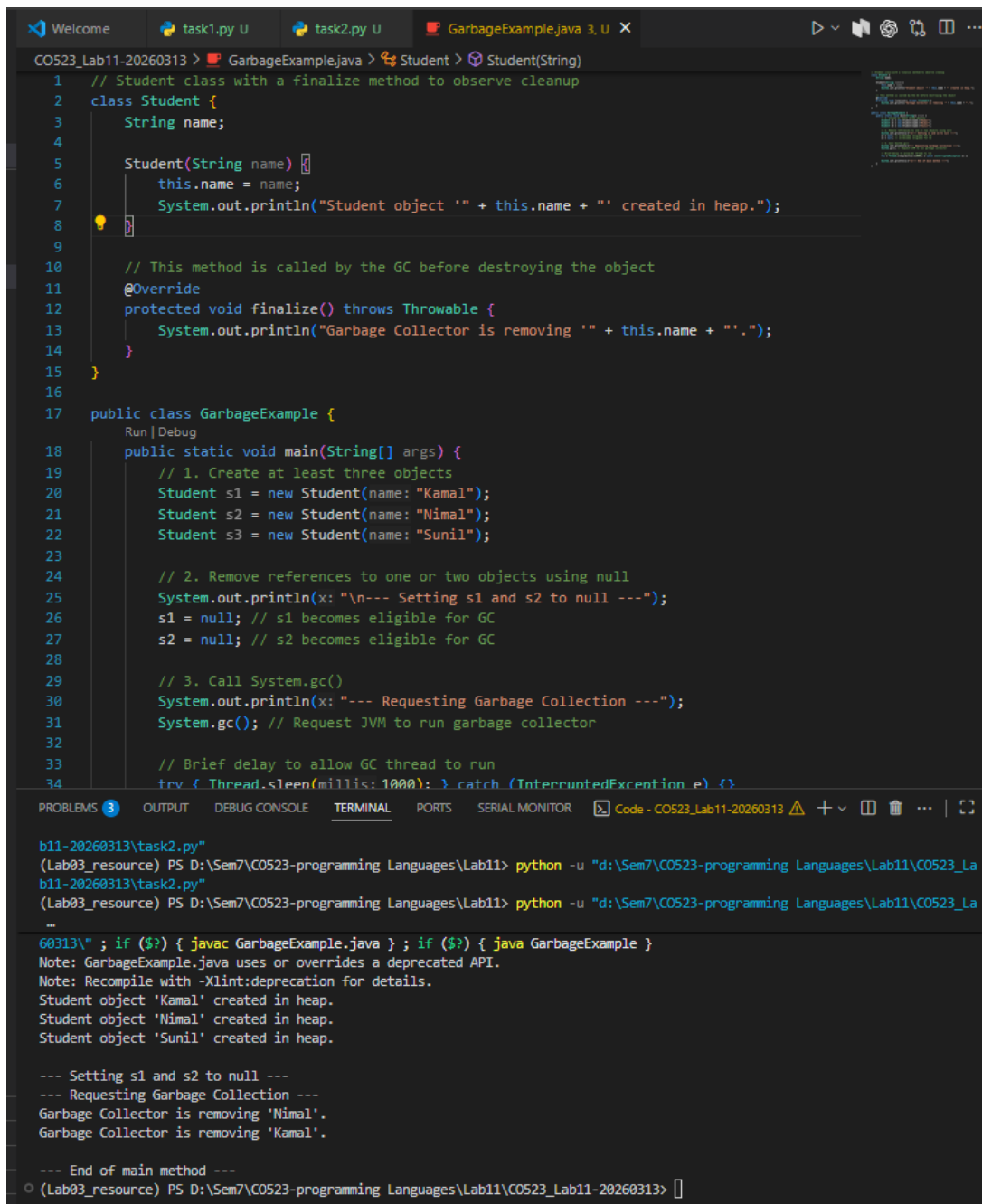
// Brief delay to allow GC thread to run
try { Thread.sleep(1000); } catch (InterruptedException e) {}

System.out.println("\n--- End of main method ---");
}
}

```

Explanation of Steps

- **Heap Allocation:** We created three objects in the heap. Variables s1, s2, and s3 stored the references (memory addresses) to these objects.
- **Eligibility:** By setting s1 and s2 to null, we ensured no references point to those objects, making them eligible for garbage collection.
- **Requesting GC:** We called System.gc(), which is a request to the JVM to perform a collection. Unlike Python's gc.collect(), Java's GC is non-deterministic, meaning the JVM decides exactly when to execute it.



```
CO523_Lab11-20260313 > GarbageExample.java > Student > Student(String)
1 // Student class with a finalize method to observe cleanup
2 class Student {
3     String name;
4
5     Student(String name) {
6         this.name = name;
7         System.out.println("Student object " + this.name + " created in heap.");
8     }
9
10    // This method is called by the GC before destroying the object
11    @Override
12    protected void finalize() throws Throwable {
13        System.out.println("Garbage Collector is removing " + this.name + ".");
14    }
15 }
16
17 public class GarbageExample {
18     Run | Debug
19     public static void main(String[] args) {
20         // 1. Create at least three objects
21         Student s1 = new Student(name: "Kamal");
22         Student s2 = new Student(name: "Nimal");
23         Student s3 = new Student(name: "Sunil");
24
25         // 2. Remove references to one or two objects using null
26         System.out.println(x: "\n--- Setting s1 and s2 to null ---");
27         s1 = null; // s1 becomes eligible for GC
28         s2 = null; // s2 becomes eligible for GC
29
30         // 3. Call System.gc()
31         System.out.println(x: "--- Requesting Garbage Collection ---");
32         System.gc(); // Request JVM to run garbage collector
33
34         // Brief delay to allow GC thread to run
35         try { Thread.sleep(1000); } catch (InterruptedException e) {}
36     }
37 }

PROBLEMS 3 OUTPUT DEBUG CONSOLE TERMINAL PORTS SERIAL MONITOR Code - CO523_Lab11-20260313 + - | | | |
b11-20260313\task2.py
(Lab03_resource) PS D:\Sem7\CO523-programming Languages\Lab11> python -u "d:\Sem7\CO523-programming Languages\Lab11\CO523_La
b11-20260313\task2.py"
(Lab03_resource) PS D:\Sem7\CO523-programming Languages\Lab11> python -u "d:\Sem7\CO523-programming Languages\Lab11\CO523_La
...
60313" ; if ($?) { javac GarbageExample.java } ; if ($?) { java GarbageExample }
Note: GarbageExample.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.
Student object 'Kamal' created in heap.
Student object 'Nimal' created in heap.
Student object 'Sunil' created in heap.

--- Setting s1 and s2 to null ---
--- Requesting Garbage Collection ---
Garbage Collector is removing 'Nimal'.
Garbage Collector is removing 'Kamal'.

--- End of main method ---
(Lab03_resource) PS D:\Sem7\CO523-programming Languages\Lab11\CO523_Lab11-20260313>
```

Output Explanation

- We successfully created three Student objects in the heap memory for "Kamal", "Nimal", and "Sunil". The terminal confirms their creation as each constructor call executed successfully.
- By setting `s1 = null` and `s2 = null`, we removed the references to the first two objects. At this point, "Kamal" and "Nimal" became eligible for garbage collection because no active references were pointing to them.

- When we called `System.gc()`, we sent a request to the Java Virtual Machine (JVM) to run the garbage collector.
- Unlike Python, which usually cleans up immediately, Java's GC is a separate background process. In this specific run, the JVM accepted the request and executed the `finalize()` method for "Nimal" and "Kamal" before reclaiming their memory.
- We can see that "Nimal" was removed before "Kamal" even though "Kamal" was set to null first. This confirms that the JVM does not follow a strict order (like LIFO) when cleaning the heap; it manages the process based on its internal algorithms.
- The object "Sunil" was not removed during the GC cycle because the reference `s3` was still active, making the object reachable and therefore not eligible for collection.

Task 03 Question : Explain whether garbage collection occurs immediately and why.

We observed that garbage collection does not necessarily occur immediately after the `System.gc()` call. In Java, `System.gc()` serves as a "hint" or a request to the JVM rather than a command. The JVM's garbage collector runs as a low-priority background thread. Because the JVM prioritizes application performance, it may delay the collection until it determines that memory is low or that there is an idle period, making the timing of object destruction unpredictable.

Task 04

Implementation Code

```
class DataObject {
    String type;

    DataObject(String type) {
        this.type = type;
        System.out.println("Heap: DataObject (" + type + ") allocated in
memory.");
    }

    @Override
    protected void finalize() {
        System.out.println("Heap: DataObject (" + type + ") is being
reclaimed.");
    }
}

public class MemoryComparison {
    public static void main(String[] args) {
```

```

        System.out.println("--- Entering main method (Stack Frame created) ---");
        processData();
        System.out.println("--- Returned to main method ---");

        // Suggesting GC to see the heap object cleanup
        System.gc();
    }

    static void processData() {
        System.out.println("--- Entering processData() ---");

        // 1. Stack Variable: Local primitive
        int stackValue = 100;
        System.out.println("Stack: local variable 'stackValue' (" + stackValue +
            ") created.");

        // 2. Heap Object: Created inside the function
        DataObject heapObj = new DataObject("DynamicData");

        System.out.println("--- Exiting processData() ---");
    } // stackValue and the reference heapObj are destroyed here
}

```

Explanation of Steps

- When we call `processData()`, a new stack frame is pushed onto the stack.
- The integer `stackValue` is stored directly within that stack frame.
- The new `DataObject()` instruction allocates space in the heap, while the variable `heapObj` on the stack merely holds a reference (address) to that heap location.
- As soon as `processData()` finishes, its stack frame is popped, and `stackValue` is destroyed instantly. The `DataObject` remains in the heap but becomes eligible for garbage collection because its reference (`heapObj`) is gone.

```
task1.py U task2.py U GarbageExample.java 3, U MemoryComparison.java 1, U X
CO523_Lab11-20260313 > MemoryComparison.java > MemoryComparison > main(String[])
1 class DataObject {
2     String type;
3
4     DataObject(String type) {
5         this.type = type;
6         System.out.println("Heap: DataObject (" + type + ") allocated in memory.");
7     }
8
9     @Override
10    protected void finalize() {
11        System.out.println("Heap: DataObject (" + type + ") is being reclaimed.");
12    }
13 }
14
15 public class MemoryComparison {
16     Run | Debug
17     public static void main(String[] args) {
18         System.out.println(x: "--- Entering main method (Stack Frame created) ---");
19         processData();
20         System.out.println(x: "--- Returned to main method ---");
21
22         // Suggesting GC to see the heap object cleanup
23         System.gc();
24
25     static void processData() {
26         System.out.println(x: "--- Entering processData() ---");
27
28         // 1. Stack Variable: Local primitive
29         int stackValue = 100;
30         System.out.println("Stack: local variable 'stackValue' (" + stackValue + ")
31         created.");
32
33         // 2. Heap Object: Created inside the function
34         DataObject heapObj = new DataObject("DynamicData");
35     }
36 }
37
38 PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL PORTS SERIAL MONITOR Code - CO523_Lab11-20260313
b11-20260313\task2.py"
(Lab03_resource) PS D:\Sem7\CO523-programming Languages\Lab11> python -u "d:\Sem7\CO523-programming Languages\Lab11\CO523_La
b11-20260313\task2.py"
(Lab03_resource) PS D:\Sem7\CO523-programming Languages\Lab11> python -u "d:\Sem7\CO523-programming Languages\Lab11\CO523_La
...
Garbage Collector is removing 'Kamal'.
--- End of main method ---
(Lab03_resource) PS D:\Sem7\CO523-programming Languages\Lab11\CO523_Lab11-20260313> cd "d:\Sem7\CO523-programming Languages\
Lab11\CO523_Lab11-20260313\" ; if ($?) { javac MemoryComparison.java } ; if ($?) { java MemoryComparison }
Note: MemoryComparison.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.
--- Entering main method (Stack Frame created) ---
--- Entering processData() ---
Stack: local variable 'stackValue' (100) created.
Heap: DataObject (DynamicData) allocated in memory.
--- Exiting processData() ---
--- Returned to main method ---
Heap: DataObject (DynamicData) is being reclaimed.
(Lab03_resource) PS D:\Sem7\CO523-programming Languages\Lab11\CO523_Lab11-20260313>
```

Explanation of Output

We observed the following execution flow, which clearly distinguishes between stack and heap memory management

- Upon execution, the program entered the main method, creating the initial stack frame. When processData() was called, a new stack frame was pushed onto the stack to manage that function's execution.
- Inside processData(), we saw the creation of stackValue. This local primitive is stored directly within the stack frame.
- The program then created a DataObject named "DynamicData". While the reference variable heapObj was stored on the stack, the actual object was allocated in the heap memory.
- Once processData() finished, its stack frame was automatically removed (popped). This instantly destroyed the stackValue and the reference heapObj.
- After returning to the main method, the "DynamicData" object in the heap no longer had any active references. When we invoked System.gc(), the JVM identified this unreachable object and reclaimed its memory, as confirmed by the message: "Heap: DataObject (DynamicData) is being reclaimed".

Task 04 Question : Explain which variables are stored in stack and which are stored in heap.

We can distinguish the storage locations as follows

- **Stack Storage:** The local primitive variable stackValue and the reference variable heapObj are stored in the stack. These are short-lived and are automatically removed when the function's execution flow completes.
- **Heap Storage:** The actual DataObject instance (the data itself) is stored in the heap. It remains there regardless of the function finishing, until the garbage collector identifies that no active stack references point to it anymore